

1. End-to-End System Workflow Sequence Flowchart

INTERACTIVE SEQUENCE
FLOW

01



User Search Input

User enters
"Hyderabad" into
GlassSearchBar with
300ms debounce.

UI / Debounce

02



Nominatim API

OpenStreetMap API
returns exact coords:
(17.385°N, 78.486°E).

REST Geocode

03



3D Vector Math

Spherical conversion to
Cartesian (x, y, z) on
Earth sphere radius
 $R=80$.

Trigonometry

04



60 FPS Earth Spin

Three.js smoothly
tweens camera to target
via shortest-path lerp
(1.1s).

WebGL 60 FPS

↓ Hypersonic Descent & City Portal Initialization ↓

05



Atmospheric Dive

Supersonic altitude
countdown (420km →

06



Hyderabad Portal

Full city page opens with
Charminar, Golconda
Fort, Nizam Itinerary &
Reviews.

07



Stripe Sandbox

User enters test card
(4242 4242 4242 4242)
for instant payment
authorization.

08



Supabase Receipt

Record saved to
'reservations' table +

Okm) through cloud
layers.

Warp Animation

Full City Page

Test Checkout

Reference ID (AV-XXXX)
generated.

DB Confirmed

2. Frontend Component Hierarchy Tree

REACT 18 & THREE.JS ARCHITECTURE

App.tsx (Main Spatial Stage & Root Viewport)

Coordinates state between 3D globe orbit, atmospheric plunge, search bar, and modals.

ROOT STAGE

01. TOP NAVIGATION

Header Bar

- "Reserve Access" Glowing CTA
- Currency Switcher (USD, EUR, INR...)
- Saved Bucket List Counter Drawer
- Spin & Win Promo Wheel

Navbar Actions

02. 3D ORBIT & SEARCH

Spatial3DGlobe

- Pure Three.js WebGL Earth Sphere (R=80)
- GlassSearchBar with 300ms debounce
- AtmosphericDiveTransition (420km)
- Shortest-Path Continuous 60 FPS Spin

Three.js / WebGL

03. CITY EXPERIENCE

ImmersiveCityExperience

- Dynamic Landmarks (Charminar, Golconda)
- Curated Day-by-Day Nizami Itinerary
- Live Supabase Reviews & Rating Writer
- "Back to Orbit" Navigation Button

Full City Portal

 Interactive Checkout & Lead Capture Modal 



BookingModal.tsx (3-Step Conversion Flow)

Step 1: Contact Details & Dates → Step 2: Dual-Column Stripe Sandbox Card → Step 3: Verified Reference
ID Boarding Pass

LEAD
CHECKOUT

3. Backend Database & Services Architecture

SUPABASE & SERVICE PIPELINES

LAYER 1



Frontend Services

- **destinationService.ts** : Dynamic relational query engine.
- **supabaseService.ts** : Persists bookings & leads.
- **stripeService.ts** : Test card authorization (4242...).
- **travelDataService.ts** : Nominatim & Open-Meteo API.

TypeScript Services

LAYER 2



High-Speed Cache

- In-Memory SWR Cache (5-Minute TTL): Returns 0ms instant data for repeated searches.
- Pre-seeded Local Fallback Repository: Provides 100% reliable offline data for Hyderabad, Rome, Tokyo, Agra, etc. if network is disconnected.

SWR & Offline Fallback

LAYER 3



Cloud Backend & APIs

- Supabase PostgreSQL: Normalized tables (`destinations`, `destination\_details`, `reservations`, `reviews`).
- Stripe Elements Sandbox: Test payments.
- OpenStreetMap Nominatim: Live geocoding.
- Open-Meteo: Real-time climate.

PostgreSQL & REST APIs

4. Supabase Relational Database Schema

Table

POSTGRESQL RELATIONAL TABLES

Table Name	Primary Key	Foreign Keys & JSONB Fields	Description & Purpose
destinations	city_slug (VARCHAR PK)	city_name, country, lat, lon, daily_rate, rating	Primary catalog of worldwide luxury destinations.
destination_details	id (UUID PK)	city_slug (FK), landmarks (JSONB), itinerary_days (JSONB)	Rich relational content for Hyderabad (Charminar, Golconda, Nizam Itineraries).
reservations	id (AV-XXXX PK)	user_email, full_name, dates, amount_paid, stripe_intent_id	Confirmed guest booking records and lead capture persistence.

Table Name	Primary Key	Foreign Keys & JSONB Fields	Description & Purpose
reviews	id (UUID PK)	city_id (FK), user_name, rating, comment, created_at	Real-time user feedback with dynamic average rating recalculation.